

EPIISODE 4

Generative AI Engineering

Build Production AI-Powered Applications

How LLMs Work

Prompt Engineering

Structured Outputs

Function/Tool Calling

Embeddings & RAG

AI Agents & Patterns

LangChain

Evaluation & Safety

TABLE OF CONTENTS

1 How LLMs Actually Work

- Tokenization
- Context window & budget
- Temperature & sampling

2 Calling LLM APIs

- Message roles
- Cost awareness
- Rate limits & retries

3 Prompt Engineering

- Clear instructions
- Few-shot examples
- Chain-of-thought

4 Structured Outputs

- JSON mode
- Schema-based prompting
- Validation

5 Zod Schema Validation

- Define schema
- Verify AI responses
- Retry strategies

6 Function/Tool Calling

- Tool definitions
- Deterministic actions
- Hybrid logic

7 Streaming Responses

- Token streaming
- Progressive rendering
- UX patterns

8 Error Handling in AI

- Malformed JSON
- Timeouts
- Retry mechanisms

9 Embeddings

- Vector space
- Similarity search
- Chunking strategies

10 Vector Databases

- Storing embeddings
- Querying
- Metadata filters

11 Building a RAG Pipeline

- Ingestion

- Retrieval
- Structured answers

1 2 Evaluating AI Systems

- Hallucination detection
- Confidence scores
- Observability

1 3 AI Agents

- Agent vs LLM call
- Reasoning + tools
- Multi-agent

1 4 Agent Design Patterns

- Planner-Executor
- Critic-Refiner
- Router agents

1 5 LangChain Practical

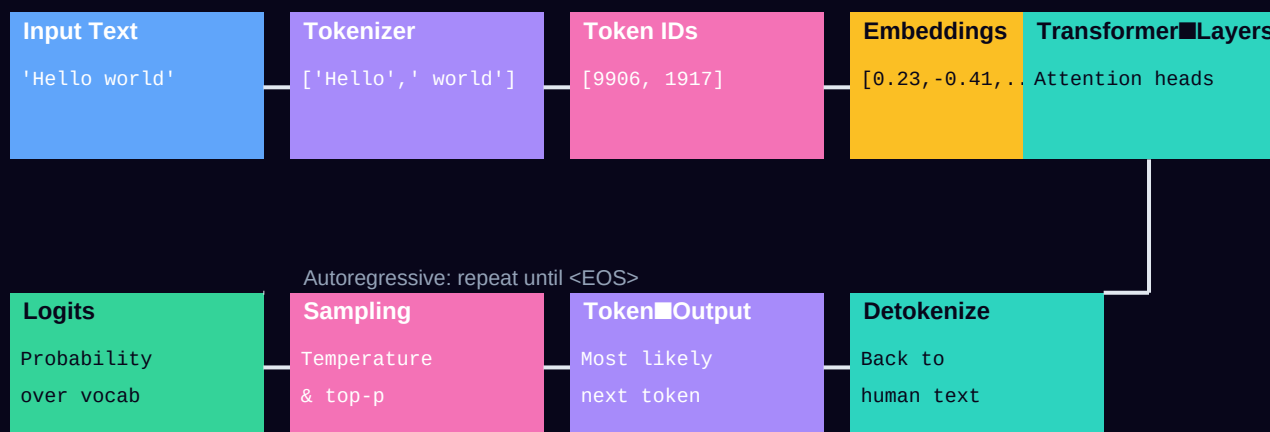
- Chains & templates
- Tools & memory
- LCEL

CHAPTER 1 — HOW LLMs ACTUALLY WORK

Inside the Black Box

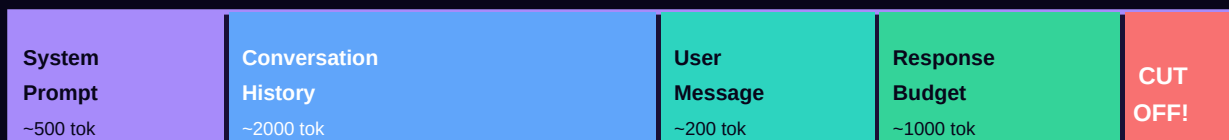
Large Language Models don't 'know' things — they PREDICT the next most likely token given everything that came before. They are trained on vast text corpora to learn statistical patterns of language. Understanding this is fundamental to using them effectively.

How LLMs Process Text — Tokenization to Output



LLM processing: text → tokens → embeddings → transformer layers → probability distribution → output token

Context Window & Token Budget

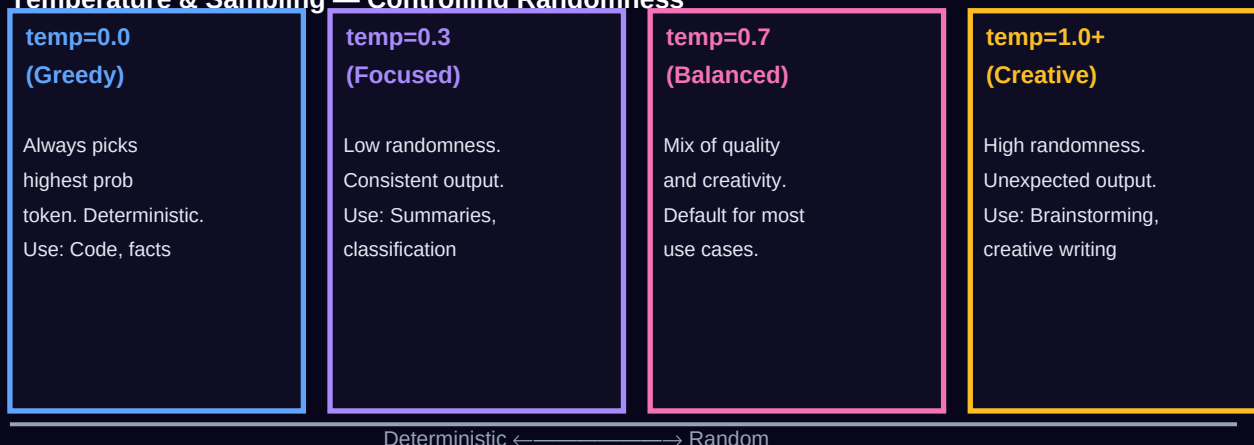


← Total Context Window (e.g. 128,000 tokens for Claude) →

Tip: 1 token ≈ 4 chars in English. 'Hello world' = 2 tokens. Code is denser.

Context window: everything the model 'sees' at once — system + history + user + response budget

Temperature & Sampling — Controlling Randomness



Temperature controls randomness — 0 is deterministic, 1+ is creative/unpredictable

Critical Mental Model for AI Engineers

- LLMs predict the NEXT TOKEN — they don't reason, retrieve, or verify.
- Same prompt can give different outputs each run (unless temp=0).
- 'Hallucination' is just confident wrong prediction — it's baked into the architecture.
- Context window is the model's entire 'working memory' — it forgets between calls.
- Tokens ≠ words: 'ChatGPT' = 1 token, 'unbelievable' = 3 tokens.
- Pricing is per token (input + output) — bad prompts literally cost money.

CHAPTER 2 — CALLING LLM APIs PROPERLY

API Message Roles

Role	Purpose	Example Content
system	Sets model behavior, persona, constraints	You are a helpful code reviewer. Always suggest tests.
user	The human's input or question	Review this function: function add(a,b) { return a+b }
assistant	Model's previous response (for multi-turn)	The function looks correct. Consider adding edge cases...

```
// Anthropic Claude API — proper structure
const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'x-api-key': process.env.ANTHROPIC_API_KEY,
    'anthropic-version': '2023-06-01'
  },
  body: JSON.stringify({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 1024,
    system: 'You are an expert code reviewer. Be concise and actionable.',
    messages: [
      { role: 'user', content: 'Review this: ' + code }
    ]
  })
});

const data = await response.json();
const text = data.content[0].text;

// Token usage tracking
console.log(`Input: ${data.usage.input_tokens} tokens`);
console.log(`Output: ${data.usage.output_tokens} tokens`);
// Cost = (input_tokens * input_price + output_tokens * output_price) / 1_000_000

// Rate limit handling — exponential backoff
async function callWithRetry(fn, maxRetries=3) {
  for (let i=0; i<maxRetries; i++) {
    try { return await fn(); }
    catch (err) {
      if (err.status === 429) {
        await new Promise(r => setTimeout(r, 2**i * 1000)); // 1s, 2s, 4s
      } else throw err;
    }
  }
  throw new Error('Max retries exceeded');
}
```

CHAPTER 3 — PROMPT ENGINEERING

The Art & Science of Prompting

Anatomy of a Well-Structured Prompt

SYSTEM	Role & persona Context & constraints Output format rules	~200 tok
FEW-SHOT EXAMPLES	Input: X → Output: Y Input: A → Output: B	~150 tok
USER INPUT	The actual task or question here	variable
OUTPUT FORMAT	JSON schema / XML or plain text spec	~50 tok

Anatomy of a well-structured prompt: each section serves a specific purpose

Prompt Engineering Techniques

```
// ■ BAD: Vague, no constraints, no format
const bad = `Tell me about this user profile: ${JSON.stringify(profile)}`;

// ■ GOOD: Clear role, specific task, format specified
const good = `
You are an expert dating profile analyst.
Analyze the following profile for compatibility with our target demographic.

Profile: ${JSON.stringify(profile)}

Provide your analysis in this EXACT JSON format:
{
  "score": <number 0-100>,
  "strengths": [<string>, ...],
  "improvements": [<string>, ...],
  "opener_suggestions": [<string>, ...]
}

Be specific, actionable, and honest. Score ruthlessly.
`;

// Few-shot prompting — teach by example
const fewShot = `
Classify sentiment as: positive, negative, or neutral.

Input: 'The product exceeded my expectations!'
Output: positive

Input: 'It was fine, nothing special'
Output: neutral

Input: 'Absolute waste of money'
Output: negative

Input: '${userInput}'
Output:
`;
```

Prompt Engineering Golden Rules

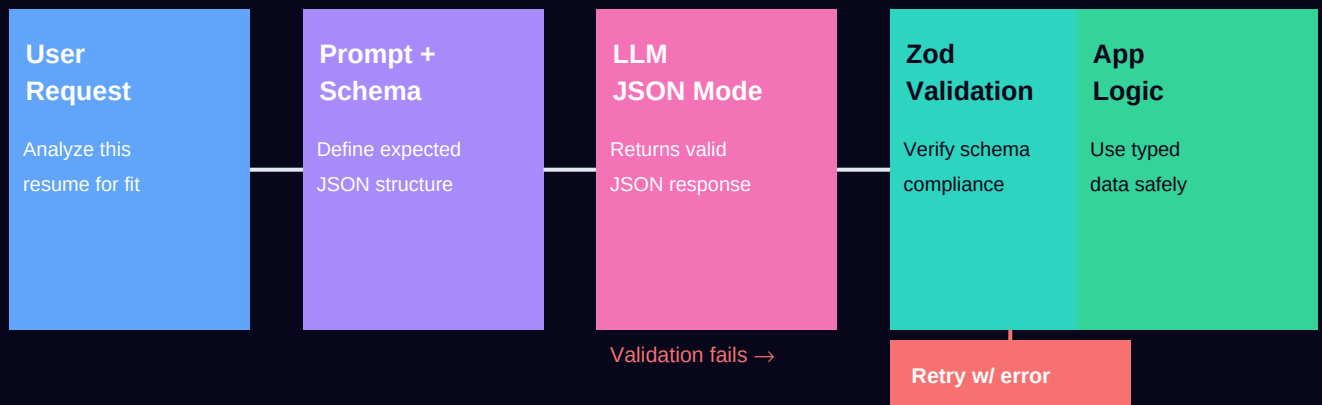
1. Be SPECIFIC: 'Summarize in 3 bullet points' beats 'summarize this'.
2. Show, don't just tell: few-shot examples outperform detailed instructions.
3. Specify output format EXPLICITLY — JSON, markdown, plain text, length.
4. Use XML/delimiter tags to separate sections: <context>, <task>, <format>.
5. Ask for step-by-step reasoning on complex tasks (chain-of-thought).
6. Test at temp=0 first — add temperature only when creativity is needed.
7. Iterate: prompt engineering is empirical — measure, tweak, measure again.

CHAPTERS 4 & 5 — STRUCTURED OUTPUTS & ZOD VALIDATION

Making AI Outputs Programmable

Unstructured text responses are hard to use in real systems. By forcing JSON output and validating with Zod, you get type-safe, predictable AI responses you can trust.

Structured Output Pipeline



Always validate AI output — never trust raw LLM responses in production code

```
import { z } from 'zod';

// 1. Define strict schema
const profileAnalysisSchema = z.object({
  compatibility_score: z.number().min(0).max(100),
  strengths:          z.array(z.string()).min(1).max(5),
  weaknesses:         z.array(z.string()).min(1).max(5),
  suggested_openers:  z.array(z.string()).min(3).max(3),
  red_flags:          z.array(z.string()),
});

type ProfileAnalysis = z.infer<typeof profileAnalysisSchema>;

// 2. Prompt the model to return JSON
async function analyzeProfile(profile: Profile): Promise<ProfileAnalysis> {
  const prompt = `Analyze this profile and return ONLY valid JSON matching:
${JSON.stringify(profileAnalysisSchema.shape, null, 2)}`;

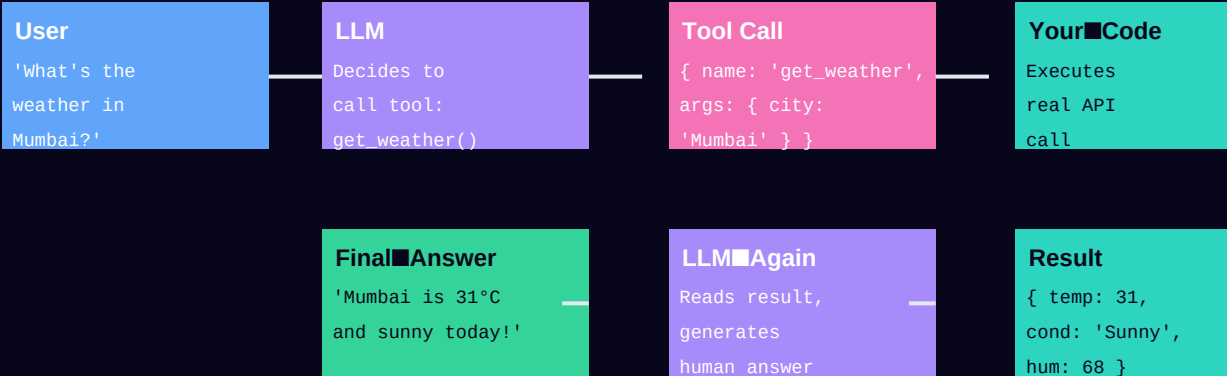
  for (let attempt = 0; attempt < 3; attempt++) {
    const raw = await callLLM(prompt);

    // 3. Parse & validate
    try {
      const json  = JSON.parse(raw);
      const result = profileAnalysisSchema.parse(json); // throws if invalid
      return result; // type-safe!
    } catch (err) {
      if (attempt === 2) throw new Error('AI returned invalid structure after 3 tries');
      // Retry with error context
      prompt += `\n\nPrevious attempt failed: ${err.message}. Fix and retry.`;
    }
  }
  throw new Error('Unreachable');
}
```

CHAPTER 6 — FUNCTION / TOOL CALLING

Giving LLMs Real-World Capabilities

Function/Tool Calling Flow



Tool calling: LLM decides WHICH function to call, your code actually executes it

```

// Define tools the LLM can call
const tools = [
  {
    name: 'get_weather',
    description: 'Get current weather for a city',
    input_schema: {
      type: 'object',
      properties: {
        city: { type: 'string', description: 'City name' },
        units: { type: 'string', enum: ['celsius', 'fahrenheit'] }
      },
      required: ['city']
    }
  },
  {
    name: 'search_web',
    description: 'Search the web for current information',
    input_schema: {
      type: 'object',
      properties: { query: { type: 'string' } },
      required: ['query']
    }
  }
];

// Actual function implementations
const toolHandlers = {
  get_weather: async ({ city, units='celsius' }) => {
    const res = await fetch(`/api/weather?city=${city}&units=${units}`);
    return res.json();
  },
  search_web: async ({ query }) => {
    const res = await fetch(`/api/search?q=${encodeURIComponent(query)}`);
    return res.json();
  }
};

// Agent loop
async function agentLoop(userMessage: string) {
  const messages = [{ role: 'user', content: userMessage }];

  while (true) {
    const response = await callLLM({ messages, tools });

    if (response.stop_reason === 'end_turn') {
      return response.content[0].text; // final answer
    }

    // LLM wants to use a tool
    const toolUse = response.content.find(b => b.type === 'tool_use');
    const result = await toolHandlers[toolUse.name](toolUse.input);

    // Feed result back to LLM
    messages.push({ role: 'assistant', content: response.content });
    messages.push({ role: 'user', content: [{
      type: 'tool_result', tool_use_id: toolUse.id,
      content: JSON.stringify(result)
    }]});
  }
}

```

CHAPTERS 7 & 8 — STREAMING & ERROR HANDLING

Streaming for Better UX

Streaming sends tokens as they're generated instead of waiting for the full response. This dramatically improves perceived performance — users see output in <500ms instead of waiting 5+ seconds.

```
// Server-side streaming (Node.js + Express)
app.get('/api/chat/stream', async (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');

  const stream = await anthropic.messages.stream({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 1024,
    messages: [{ role: 'user', content: req.query.message }]
  });

  for await (const event of stream) {
    if (event.type === 'content_block_delta') {
      res.write(`data: ${JSON.stringify({ text: event.delta.text })}\n\n`);
    }
  }

  res.write('data: [DONE]\n\n');
  res.end();
});

// Client-side: consuming the stream
const response = await fetch('/api/chat/stream?message=' + encodeURIComponent(msg));
const reader = response.body.getReader();
const decoder = new TextDecoder();

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  const text = decoder.decode(value);
  const lines = text.split('\n').filter(l => l.startsWith('data:'));

  for (const line of lines) {
    const data = line.replace('data: ', '');
    if (data === '[DONE]') return;
    const { text } = JSON.parse(data);
    appendToUI(text); // update UI incrementally
  }
}
```

AI-Specific Error Handling

```
class AIError extends Error {
  constructor(type, message, retryable=false) {
    super(message); this.type=type; this.retryable=retryable;
  }
}

async function safeAICall(prompt, schema) {
  try {
    const raw = await callLLM(prompt);

    // Validate JSON structure
    let parsed;
    try { parsed = JSON.parse(raw); }
    catch { throw new AIError('INVALID_JSON', 'Model returned non-JSON', true); }

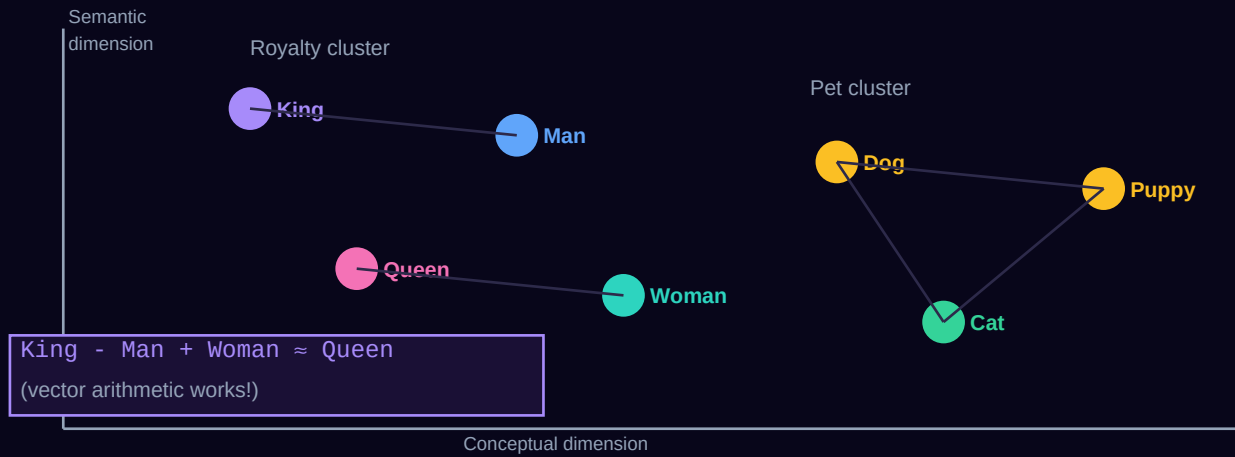
    // Validate schema
    const result = schema.safeParse(parsed);
    if (!result.success) throw new AIError('SCHEMA_MISMATCH', result.error.message, true);

    return result.data;
  } catch (err) {
    if (err.status === 429) throw new AIError('RATE_LIMIT', 'Too many requests', true);
    if (err.status === 529) throw new AIError('OVERLOADED', 'API overloaded', true);
    throw err;
  }
}
```

CHAPTERS 9 & 10 — EMBEDDINGS & VECTOR DATABASES

Semantic Search with Embeddings

Embeddings — Meaning in Vector Space



Embeddings: words/sentences mapped to high-dimensional vectors — similar meaning = close vectors

```
// Generate embeddings with OpenAI
const { OpenAI } = require('openai');
const openai = new OpenAI();

async function embed(text) {
  const res = await openai.embeddings.create({
    model: 'text-embedding-3-small', // 1536 dimensions
    input: text,
  });
  return res.data[0].embedding; // array of 1536 floats
}

// Cosine similarity – measure how close two vectors are
function cosineSimilarity(a, b) {
  const dot = a.reduce((s, ai, i) => s + ai*b[i], 0);
  const magA = Math.sqrt(a.reduce((s, ai) => s + ai*ai, 0));
  const magB = Math.sqrt(b.reduce((s, bi) => s + bi*bi, 0));
  return dot / (magA * magB); // 1=identical, 0=unrelated, -1=opposite
}

// Store in Pinecone (vector DB)
const { Pinecone } = require('@pinecone-database/pinecone');
const pc = new Pinecone({ apiKey: process.env.PINECONE_KEY });
const index = pc.index('my-docs');

// Upsert vectors
await index.upsert([
  {
    id: 'doc-1-chunk-3',
    values: await embed(chunkText),
    metadata: { source: 'handbook.pdf', page: 3, text: chunkText }
  },
  {
    id: 'doc-1-chunk-4',
    values: await embed(chunkText2),
    metadata: { source: 'handbook.pdf', page: 4, text: chunkText2 }
  }
]);

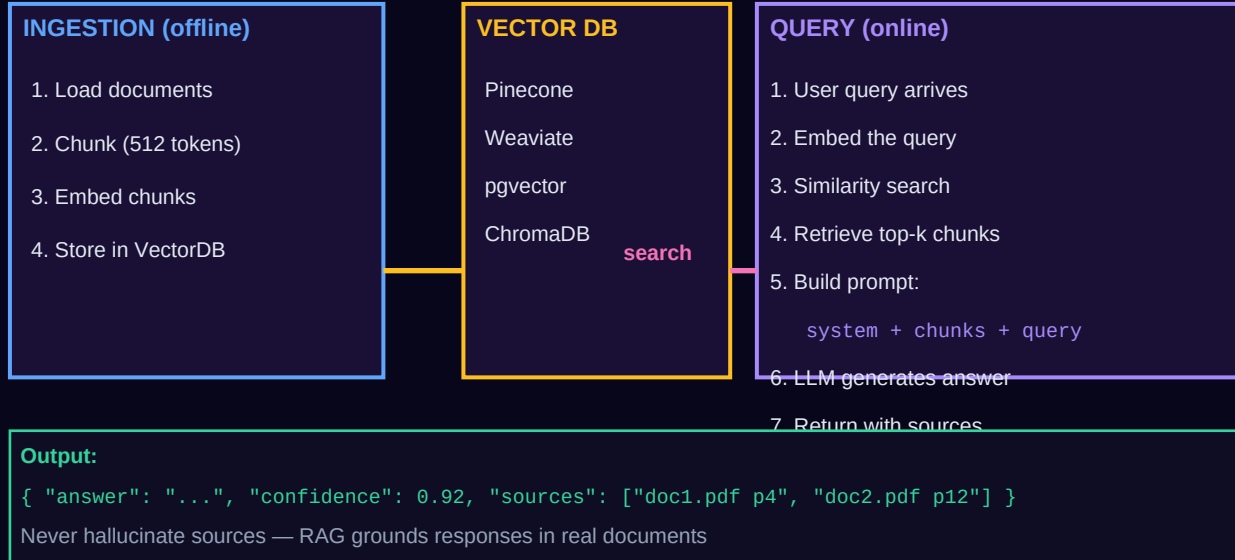
// Query – semantic search
const queryEmbedding = await embed('How do I request vacation days?');
const results = await index.query({
  vector: queryEmbedding,
  topK: 5,
  filter: { source: 'handbook.pdf' }, // metadata filter
  includeMetadata: true,
});

const chunks = results.matches.map(m => m.metadata.text);
```


CHAPTER 11 — BUILDING A RAG PIPELINE

Retrieval Augmented Generation

RAG Pipeline — Retrieval Augmented Generation



RAG grounds LLM responses in real documents — eliminates hallucination on domain knowledge

```
// Complete RAG implementation
async function ragQuery(userQuestion: string) {
  // 1. Embed the question
  const queryVec = await embed(userQuestion);

  // 2. Retrieve relevant chunks
  const results = await vectorIndex.query({
    vector: queryVec, topK: 5, includeMetadata: true
  });

  const context = results.matches
    .filter(m => m.score > 0.75) // only high-confidence matches
    .map(m => m.metadata.text).join('\n\n');

  if (!context) return { answer: 'No relevant information found.', sources: [] };

  // 3. Build grounded prompt
  const prompt = `
  You are a helpful assistant. Answer ONLY using the provided context.
  If the answer isn't in the context, say 'I don't have that information.'

  Context:
  ${context}

  Question: ${userQuestion}

  Respond in JSON: { answer: string, confidence: number, key_points: string[] }
  `;

  // 4. Generate grounded answer
  const raw = await callLLM(prompt);
  const parsed = JSON.parse(raw);

  // 5. Return with sources
  return {
    ...parsed,
    sources: results.matches.map(m => ({
      document: m.metadata.source,
      page: m.metadata.page,
      score: m.score
    })))
  };
}

// Chunking strategy – critical for retrieval quality
function chunkDocument(text: string, chunkSize=512, overlap=50) {
  const words = text.split(' ');
  const chunks = [];
  for (let i=0; i<words.length; i+=chunkSize-overlap) {
    chunks.push(words.slice(i, i+chunkSize).join(' '));
  }
  return chunks;
}
```

CHAPTER 12 — EVALUATING AI SYSTEMS

Why LLMs Hallucinate & How to Evaluate

Why LLMs Hallucinate & How to Detect			
Why It Happens Predicts plausible text, not 'truth' No internal DB of facts	Common Types Made-up citations Fake code APIs Wrong dates/stats Confident lies	Detection Methods Ask for sources Verify against DB Consistency check Confidence score	Prevention RAG grounding Few-shot examples Schema validation Human-in-loop

Know your failure modes — evaluation is as important as building the system

```
// Confidence scoring – ask the model to self-evaluate
const evalPrompt = `
  You just answered: '${answer}'
  Question was: '${question}'
  Context used: '${context}'

  Rate your answer confidence (0.0-1.0) based on:
  - Is it fully supported by the context?
  - Are there any uncertainties or gaps?

  Return JSON: { confidence: number, reasoning: string }
`;

// Hallucination detection – cross-check claims
async function detectHallucination(answer, context) {
  const claims = await extractClaims(answer); // split into verifiable facts

  const checks = await Promise.all(claims.map(async claim => {
    const checkPrompt = `Is '${claim}' supported by this context: '${context}'?`;
    Return JSON: { supported: boolean, evidence: string }`;
    const result = JSON.parse(await callLLM(checkPrompt));
    return { claim, ...result };
  }));

  const unsupported = checks.filter(c => !c.supported);
  return { hallucination_count: unsupported.length, details: unsupported };
}

// Logging AI calls for observability
async function loggedAICall(prompt, metadata) {
  const start = Date.now();
  const result = await callLLM(prompt);

  await db.aiLog.create({
    prompt_hash: hash(prompt),
    model: 'claude-sonnet-4-20250514',
    input_tokens: result.usage.input_tokens,
    output_tokens: result.usage.output_tokens,
    latency_ms: Date.now()-start,
    metadata // user_id, feature, experiment
  });
  return result;
}
```

CHAPTERS 13 & 14 — AI AGENTS & DESIGN PATTERNS

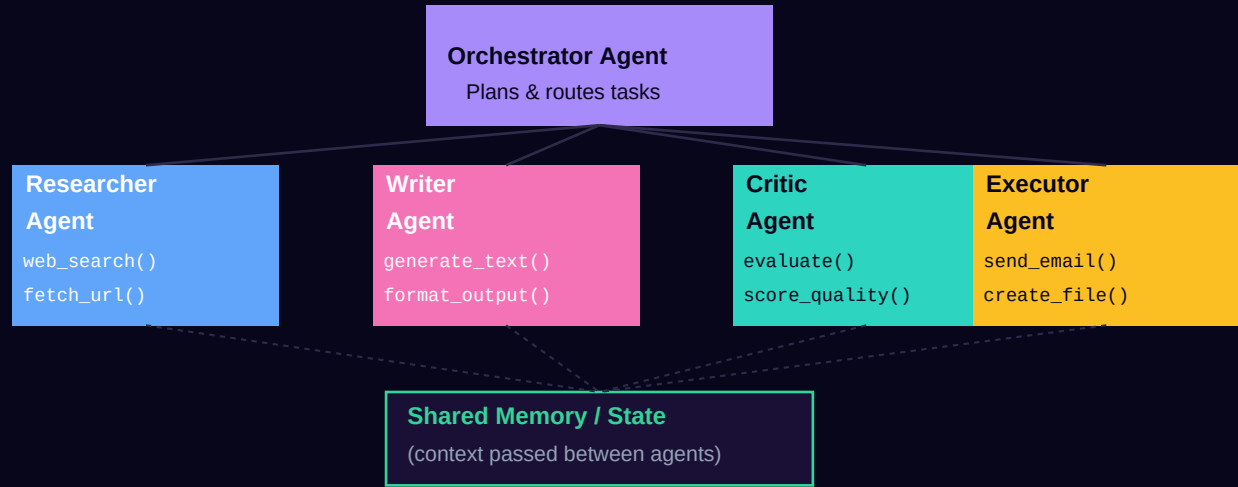
When to Use Agents

AI Agent vs Single LLM Call

Single LLM Call	AI Agent
User → Prompt → LLM → Answer	User Goal
✓ Simple & fast	→ LLM (plan next step)
✓ Cheap & predictable	→ Tool Call (web, code, DB)
✗ No tool use	→ Observe result
✗ No multi-step reasoning	→ LLM (next step?)
✗ Can't verify own output	→ Tool Call again...
	→ LLM (done?) → Answer
Use for: simple Q&A, summarization, classification	✓ Autonomous multi-step reasoning
	✗ Expensive & slower & harder to debug

Agents add power but also complexity, cost, and latency — use only when needed

Multi-Agent Architecture Pattern



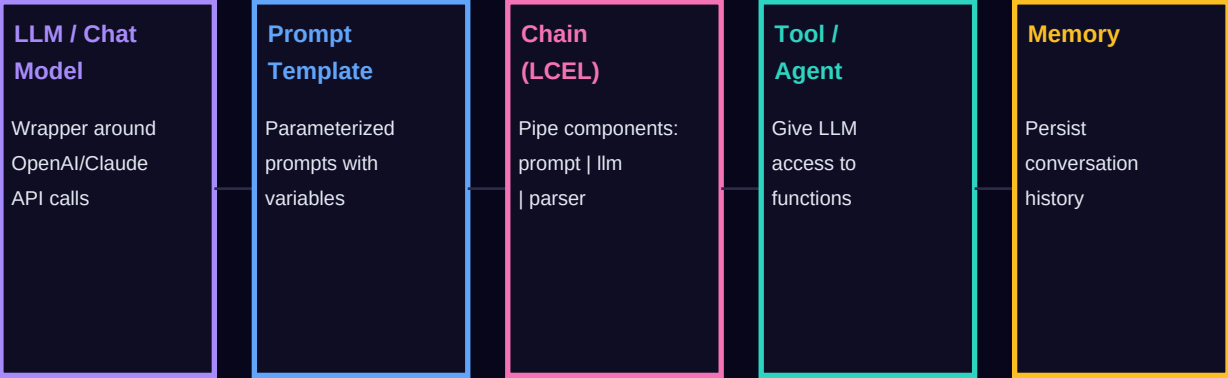
Multi-agent: orchestrator delegates to specialized sub-agents with shared memory

Pattern	Structure	Best For	Watch Out For
Planner→Executor	Planner creates steps, Executor runs them	Complex multi-step tasks	Planner hallucinating bad plans
Researcher→Writer	Research gathers info, Writer synthesizes	Content generation	Context window limits
Critic→Refiner	Critic scores output, Refiner improves	Quality-sensitive outputs	Infinite refinement loops
Router Agent	LLM decides which specialist agent to call	Mixed-task systems	Misrouting edge cases
Parallel Agents	Multiple agents run simultaneously	Independent subtasks	State synchronization

CHAPTER 15 — LANGCHAIN PRACTICAL

Building with LangChain

LangChain Core Concepts



LangChain: composable building blocks for LLM applications

```
import { ChatAnthropic }      from '@langchain/anthropic';
import { ChatPromptTemplate }  from '@langchain/core/prompts';
import { StringOutputParser }  from '@langchain/core/output_parsers';
import { RunnableSequence }    from '@langchain/core/runnables';

const llm    = new ChatAnthropic({ model: 'claude-sonnet-4-20250514' });
const parser = new StringOutputParser();

// LCEL Chain — pipe operator
const prompt = ChatPromptTemplate.fromMessages([
  ['system', 'You are a {role}. Answer in {language}.'],
  ['human',  '{question}'],
]);

const chain = prompt.pipe(llm).pipe(parser);

const answer = await chain.invoke({
  role: 'senior backend engineer',
  language: 'simple English',
  question: 'What is a database index?'
});

// RAG chain with retriever
import { createRetrievalChain } from 'langchain/chains/retrieval';
import { createStuffDocumentsChain } from 'langchain/chains/combine_documents';

const docChain = await createStuffDocumentsChain({ llm, prompt: ragPrompt });
const ragChain = await createRetrievalChain({ retriever, combineDocsChain: docChain });

const result = await ragChain.invoke({ input: 'What is our refund policy?' });
// result.answer, result.context (source documents)

// Tool-using agent
import { createReactAgent } from 'langchain/agents';
import { DuckDuckGoSearch } from '@langchain/community/tools/duckduckgo_search';

const agent = await createReactAgent({
  llm, tools: [new DuckDuckGoSearch()],
  prompt: agentPrompt,
});

const agentExecutor = new AgentExecutor({ agent, tools });
const res = await agentExecutor.invoke({ input: 'Latest React version?' });
```

EPISODE 4 COMPLETE — WHAT YOU'VE LEARNED

✓ How LLMs Work	Tokenization, context window, temperature, probabilistic output
✓ LLM APIs	Message roles, token budgeting, rate limiting, retries
✓ Prompt Engineering	Specificity, few-shot, output format, iteration
✓ Structured Outputs	JSON mode, schema-based prompting, type safety
✓ Zod Validation	Schema definition, AI response verification, retry logic

✓ Function Calling	Tool definitions, agent loop, hybrid LLM+code
✓ Streaming	SSE, token streaming, progressive UI rendering
✓ Error Handling	Malformed JSON, timeouts, retry strategies, logging
✓ Embeddings	Vector space, cosine similarity, semantic search
✓ Vector Databases	Pinecone/Weaviate, upsert, filtered queries
✓ RAG Pipeline	Chunking, ingestion, retrieval, grounded prompting
✓ Evaluation	Hallucination detection, confidence scoring, observability
✓ AI Agents	Agent loop, tool calling, when NOT to use agents
✓ Design Patterns	Planner-Executor, Critic-Refiner, Router agents
✓ LangChain	LCEL chains, RAG chains, agent executor

What's Next — Episode 5

Episode 5: Production Systems, Infrastructure & Deployment

- Linux basics, SSH, DevOps foundations
- Docker & containerization, writing Dockerfiles
- AWS: EC2, Security Groups, Load Balancers, CloudFront CDN
- CI/CD with GitHub Actions — automate on git push
- Scalability patterns, Redis caching, monitoring & observability